

Today

INTRODUCTION TO COMPILERS.txt

Plain Text

give a good summary on the whole lecture in detailssss explain in detailssss

Here's a **detailed summary and explanation** of *Lecture 12 – Introduction to Compilers* by Dr. Sherine Nagi, organized phase by phase so you can easily memorize it for your exam 📌

Structure of a Compiler

A compiler has **two major parts**:

Part	Description
Front End (Analysis)	Breaks the source program into pieces, checks grammar and meaning, and builds an intermediate representation. Detects syntax and semantic errors.
Back End (Synthesis)	Uses the intermediate representation and symbol table to construct the target (machine) program.

Phases of Compilation

1 Lexical Analysis (Scanner)

- **Purpose:** Reads the source code character by character and groups them into **lexemes** (meaningful sequences).
- **Output:** Tokens of the form `<token-name, attribute-value>`.

Example:

Code

position = initial + rate * 60

becomes

- `<id,1> <=><id,2> <+> <id,3> <*> <60>`
- **Tasks:**
 - Remove comments and whitespace.
 - Produce tokens.
 - Correlate error messages with source lines.
- **Concepts:**
 - **Token:** Pair of name and attribute (e.g., `<id, pointer>`).
 - **Lexeme:** Actual character sequence matching a token pattern.
 - **Pattern:** Rule describing valid lexemes.
- **Token Categories:**
 - Keywords, operators, identifiers, constants, punctuation.
- **Attributes:** Stored in the **symbol table** (type, location, etc.).

2 Syntax Analysis (Parser)

- **Purpose:** Uses tokens to build a **syntax tree** showing grammatical structure and operation order.
- **Tool:** Context-Free Grammar (CFG) or **BNF notation**.
- **Grammar Components:**
 - Terminals (tokens)
 - Nonterminals (syntactic variables)
 - Start symbol
 - Productions (rules)

Example Grammar (Pascal-like):

Code

`<read> ::= READ(<id-list>)`

`<id-list> ::= id | <id-list>, id`

`<assign> ::= id := <exp>`

`<exp> ::= <term> | <exp> + <term> | <exp> - <term>`

`<term> ::= <factor> | <term> * <factor> | <term> DIV <factor>`

`<factor> ::= id | int | (<exp>)`

-
- **Parsing Types:**
 - **Top-down (Recursive Descent):** Builds tree from start symbol downward.

3 Semantic Analysis

- **Purpose:** Checks **meaning** and **type consistency** using the syntax tree and symbol table.
- **Tasks:**
 - Type checking (e.g., operands of **+** must match).
 - Ensures semantic correctness.
 - Stores type info for later code generation.

4 Intermediate Code Generation

- **Purpose:** Converts syntax tree into a **machine-like intermediate representation**.
- **Properties:**
 - Easy to produce.
 - Easy to translate into target code.

5 Code Optimization

- **Purpose:** Improves intermediate code for efficiency.
- **Goal:** Generate “better” target code — faster, smaller, or more efficient.

6 Code Generation

- **Purpose:** Converts optimized intermediate code into **target machine code**.

Example:

Code

t1 = id3 * 60.0

id1 = id2 + t1

becomes:

Code

LDF R2, id3

MULF R2, R2, #60.0

LDF R1, id2

ADDF R1, R1, R2

STF id1, R1

•

Tokens, Patterns, Lexemes

- **Token:** `<name, attribute>`
- **Lexeme:** Actual sequence matching a pattern.
- **Pattern:** Defines valid lexemes.

Examples

```
printf("Total = %d\n", score);
```

- → `printf` and `score` are identifiers; `"Total = %d\n"` is a literal.

```
E = M * C ** 2 (Fortran)
```

- → `<id, pointer(E)>` `<assign op>` `<id, pointer(M)>` `<id, pointer(C)>`
`<number, 2>`



Strings and Languages

- **Alphabet:** Finite set of symbols (e.g., ASCII, binary `{0, 1}`).
- **String:** Sequence of symbols.
- **Language:** Set of strings over an alphabet.

Operations

- **Union ($L \cup D$):** Combine sets.
- **Concatenation (LD):** Join strings.
- **Closure (L^*):** Zero or more repetitions.

Example:

`L(LUD)^*` → all strings starting with a letter followed by letters/digits.



Regular Expressions & Definitions

- Used to describe patterns for tokens.

Example for C identifiers:

- `letter_(letter_|digit)^*`
- Extensions add more expressive power.
- **Regular Definitions:** Assign names to regexes for reuse.

Transition Diagrams

- **States:** Represent scanning conditions.
- **Edges:** Show transitions based on input symbols.
- **Start State:** Entry point.
- **Accepting State:** Lexeme recognized.
- **Retract ():*** Move pointer back one position if needed.

Used for:

- Recognizing identifiers, reserved words, numbers, whitespace.

Syntax Analysis (Detailed)

- **Context-Free Grammar (CFG):** Defines valid syntax.
- **Parser Role:** Checks token sequence against grammar.
- **Symbols:**
 - **Terminals:** Actual tokens (e.g., `+`, `id`).
 - **Nonterminals:** Abstract constructs (e.g., `<expr>`).
 - **Start Symbol:** Entry point of grammar.
 - **Productions:** Rules combining terminals/nonterminals.

Syntax Tree

- Represents hierarchical structure of source code.
- Each node = grammar construct.
- Used for semantic analysis and code generation.

Summary of Compiler Workflow

Phase	Input	Output	Purpose
Lexical Analysis	Source code	Tokens	Breaks text into lexemes
Syntax Analysis	Tokens	Parse tree	Checks grammar

Semantic Analysis	Parse tree + symbol table	Annotated tree	Checks meaning and types
Intermediate Code	Annotated tree	Intermediate code	Machine-like representation
Optimization	Intermediate code	Improved code	Efficiency
Code Generation	Optimized code	Target machine code	Final executable

Would you like me to turn this into a **visual flowchart or slide outline** for quick exam memorization next? It would make the six phases and their relationships crystal clear.

give mroe detialsss

Perfect — let's expand your lecture summary into a **deep, exam-level explanation** that covers every concept in *Dr. Sherine Nagi's Lecture 12: Introduction to Compilers* with full technical clarity and examples.

Overview: What Is a Compiler?

A **compiler** is a system program that translates a **source program** written in a high-level language (like C, Java, or Pascal) into a **target program** (usually machine code).

It performs this translation through **two main parts**:

Part	Function	Output
Front End (Analysis)	Breaks the source program into components, checks syntax and semantics, and produces an intermediate representation.	Intermediate code + symbol table

Back End (Synthesis)	Uses intermediate code and symbol table to generate optimized machine code.	Target program
---------------------------------	---	----------------

Phases of Compilation (Detailed Breakdown)

Lexical Analysis (Scanner)

Purpose: Converts raw source code into tokens — the smallest meaningful units.

Process:

- Reads characters and groups them into **lexemes**.
- Produces **tokens** of the form `<token-name, attribute-value>`.
- Removes comments and whitespace.
- Correlates error messages with source lines.

Example:

plaintext

position = initial + rate * 60

Lexeme	Token
position	<id,1>
=	<=>
initial	<id,2>
+	<+>
rate	<id,3>
*	<*>
60	<60>



Lexeme Token

Concepts:

- **Token:** Symbolic representation (e.g., `<id, pointer>`).
- **Lexeme:** Actual sequence of characters.
- **Pattern:** Rule describing valid lexemes.

Token Categories:

1. Keywords (`if, while, return`)
2. Operators (`+, -, *, /`)
3. Identifiers (variable names)
4. Constants (numbers, strings)
5. Punctuation (`,, ;, (, )`)

Attributes: Stored in the **symbol table** (type, location, scope).

Example (C):

c

```
printf("Total = %d\n", score);
```

- `printf` and `score` → identifiers
- `"Total = %d\n"` → string literal

Example (Fortran):

fortran

```
E = M * C ** 2
```

Tokens:

```
<id,E> <assign> <id,M> <id,C> <number,2>
```

2 Syntax Analysis (Parser)

Purpose: Checks grammatical structure using **context-free grammar (CFG)** or **BNF notation**.

Role:

- Receives tokens from the lexical analyzer.
- Builds a **syntax tree** showing how tokens fit grammar rules.
- Detects syntax errors.

Grammar Components:

Component	Description
Terminals	Actual tokens (e.g., <code>+</code> , <code>id</code> , <code>number</code>)
Nonterminals	Abstract syntactic variables (e.g., <code><expr></code> , <code><stmt></code>)
Start Symbol	Entry point of grammar
Productions	Rules defining valid structures

Example Grammar (Simplified Pascal):

plaintext

`<read> ::= READ(<id-list>)`

`<id-list> ::= id | <id-list>, id`

`<assign> ::= id := <exp>`

`<exp> ::= <term> | <exp> + <term> | <exp> - <term>`

`<term> ::= <factor> | <term> * <factor> | <term> DIV <factor>`

`<factor> ::= id | int | (<exp>)`

Parsing Methods:

- **Top-down (Recursive Descent):** Starts from the start symbol and expands productions.
- **Bottom-up:** Builds from tokens upward to the start symbol.

Output: Syntax tree (hierarchical representation of program structure).

3 Semantic Analysis

Purpose: Ensures the program makes logical sense.

Tasks:

- **Type checking:** Ensures operands match operator types.
- **Scope resolution:** Confirms variable declarations are valid.
- **Consistency checks:** Verifies semantic rules (e.g., no division by string).

Example:

If `position`, `initial`, and `rate` are floats, then:

plaintext

```
position = initial + rate * 60
```

is valid.

If one were an integer, the compiler would flag a type mismatch.

Output: Annotated syntax tree with type and semantic information.

4 Intermediate Code Generation

Purpose: Converts the annotated syntax tree into a **machine-like intermediate representation**.

Properties:

- Easy to produce.
- Easy to translate into target code.

Example:

plaintext

position = initial + rate * 60

Intermediate code:

plaintext

t1 = rate * 60

t2 = initial + t1

position = t2

5 Code Optimization

Purpose: Improves intermediate code for efficiency.

Types of Optimization:

Type	Example	Effect
Constant Folding	<code>rate * 60</code> → precomputed	Reduces runtime
Dead Code Elimination	Removes unused variables	Saves memory
Strength Reduction	Replace <code>x * 2</code> with <code>x + x</code>	Faster execution
Loop Optimization	Moves invariant computations outside loops	Improves performance

Goal: Generate “better” target code — faster, smaller, and more efficient.

Code Generation

Purpose: Converts optimized intermediate code into **target machine code**.

Process:

- Allocates registers.
- Translates operations into assembly or machine instructions.
- Handles addressing and data movement.

Example:

Intermediate:

plaintext

t1 = rate * 60

t2 = initial + t1

position = t2

Target (assembly-like):

plaintext

LDF R2, rate

MULF R2, R2, #60.0

LDF R1, initial

ADDF R1, R1, R2

STF position, R1

Lexical Theory: Strings, Languages, and Regular Expressions

Alphabet

A finite set of symbols (letters, digits, punctuation).

Examples:

- Binary alphabet: $\{0, 1\}$
- ASCII: used in most systems.

String

A finite sequence of symbols from an alphabet.

- Length: $|s|$
- Empty string: ϵ

Language

A set of strings over an alphabet.

Operations on Languages

Operation	Description	Example
Union ($L \cup D$)	Combine sets	Letters $\cup$ digits
Concatenation (LD)	Join strings	$AB, a1, b2$
Closure (L^*)	Zero or more repetitions	$\{\epsilon, A, AA, AAA, \dots\}$

Example:

- $L(L \cup D)^*$ $\rightarrow$ all strings starting with a letter followed by letters/digits.
- D^+ $\rightarrow$ one or more digits.

Regular Expressions

Used to describe token patterns.

Example (C identifiers):

plaintext

letter_(letter_|digit)*

Meaning:

- Starts with a letter or underscore.
- Followed by zero or more letters or digits.

Operators:

Symbol	Meaning
,	Union
*	Zero or more
()	Grouping
Concatenation	Sequential combination

Regular Definitions:

Assign names to regexes for reuse:

plaintext

digit = [0-9]

letter = [A-Za-z]

id = letter(letter|digit)*

Transition Diagrams

Graphical representation of token recognition.

Components:

- **States:** Circles representing scanning conditions.
- **Edges:** Transitions labeled by input symbols.
- **Start State:** Entry point.
- **Accepting State:** Lexeme recognized.
- *Retract ()*:* Move pointer back one position if needed.

Used for:

- Identifiers
- Reserved words
- Numbers
- Whitespace

Example:

Transition diagram for relational operators (<, <=, <>, >).

Syntax Tree

A **tree-like structure** showing how grammar rules apply to tokens.

Each node represents a construct (expression, term, factor).

Used for:

- Semantic analysis
- Intermediate code generation

Summary Table of Compiler Phases

Phase	Input	Output	Purpose
Lexical Analysis	Source code	Tokens	Breaks text into lexemes

Syntax Analysis	Tokens	Parse tree	Checks grammar
Semantic Analysis	Parse tree + symbol table	Annotated tree	Checks meaning and types
Intermediate Code	Annotated tree	Intermediate code	Machine-like representation
Optimization	Intermediate code	Improved code	Efficiency
Code Generation	Optimized code	Target machine code	Final executable

This lecture builds the foundation for understanding **compiler design**, showing how a high-level program passes through structured transformations — from characters to executable machine code.

Would you like me to now create a **visual flowchart diagram** of